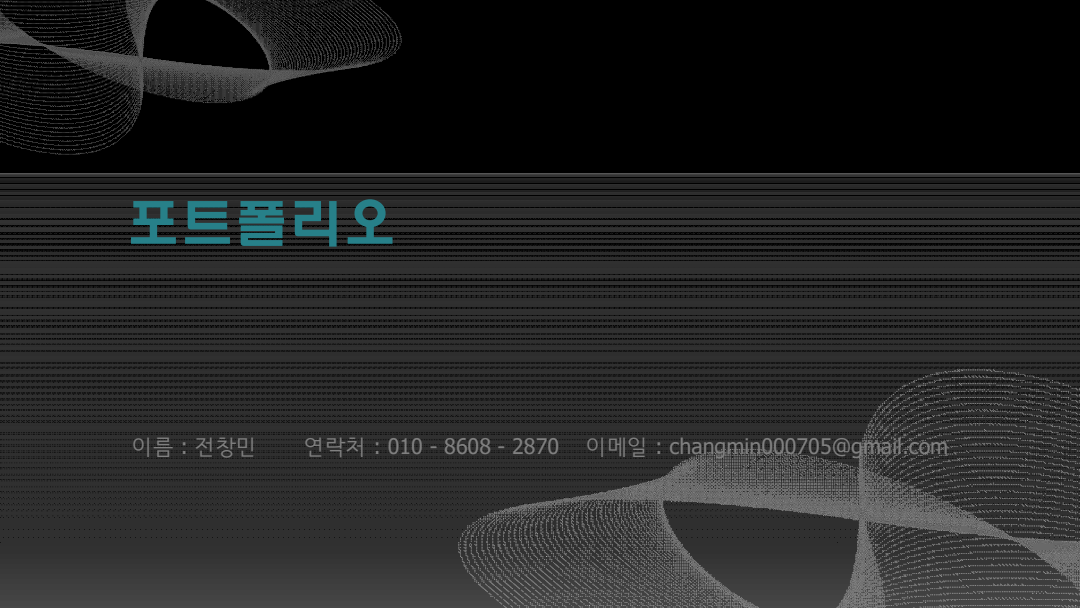




포트폴리오

이름 : 전창민

연락처 : 010 - 8608 - 2870

이메일 : changmin000705@gmail.com

목차

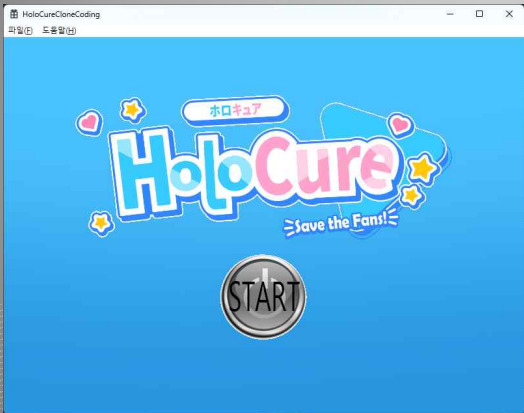
01. HoloCure Clone Project

- 프로젝트 소개
- 핵심 시스템
- 트러블 슈팅
 - 회고

02. Project Cube

- 프로젝트 소개
- 핵심 시스템
- 기술적 회고

HoloCure Clone Project



- [시연 영상](#)

- 프로젝트 소개
- HoloCure의 핵심 시스템을 WinApi와 C++로 직접 구현하며
- 게임 프레임워크와 객체 생명주기 관리 방식을 학습하기 위해 제작한 프로젝트입니다.
- 주요 구현
 - Game Loop
 - Scene Manager
 - Object Pool
 - Signal(Event)
 - Equipment
 - Collision
 - UI
- 개발 환경
 - Language : C++
 - Framework : WinAPI + GDI
- [GitHub](#)

HoloCure Clone Project

- 프로젝트 소개



● 주요 목표

- - WinAPI 기반 게임 프레임워크 제작
- - Scene 기반 객체 관리
- - Object Pool 구현
- - Signal(Event) 시스템 구현
- - UI Framework 제작

● 기술 스택

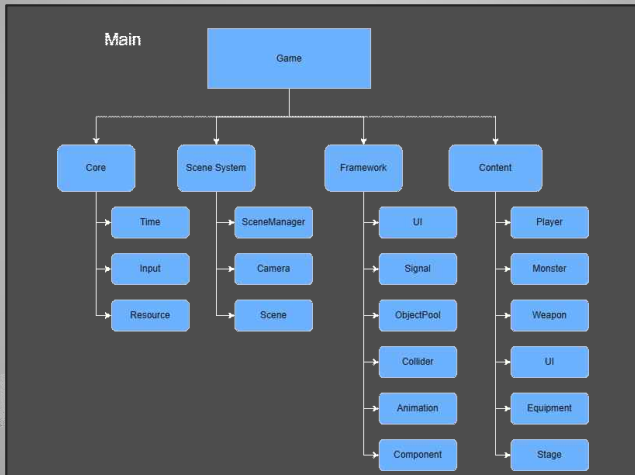
- - C++
- - WinAPI
- - GDI
- - STL

● 디자인 패턴

- - Singleton
- - Observer Pattern
- - Component Pattern
- - Object Pooling

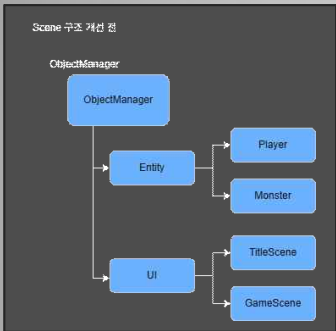
HoloCure Clone Project

- 프로젝트 구조도 - 메인 구조도



HoloCure Clone Project

- 프로젝트 구조도 - Scene System

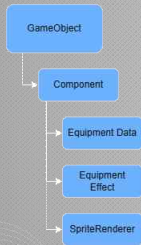


- ObjectManager 중심 구조에서 Scene 중심 구조로 변경하였다.
- Scene이 객체의 생성 / 삭제 / 생명주기를 관리하며, 씬 전환 시 불필요한 객체 순회를 방지하였다.

HoloCure Clone Project

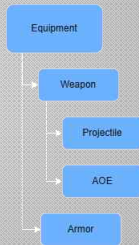
- 프로젝트 구조도 - Equipment System

Equipment 초기 설계안



- 초기 설계
- 설계의도
- Unity Component 구조를 참고하여 데이터와 기능을 조합하는 구조를 목표로 설계하였다.
- 문제
 - 장비별로 다음과 같은 차이가 존재하였다.
 - - 공격 판정(투사체 / 범위 공격)
 - - 특수 효과
 - - 레벨 효과
 - Component 조합보다 장비별 예외 처리가 계속 증가하였다.
 - 현재 프로젝트 규모에서는 Component보다 상속이 더 단순하고 관리하기 쉬웠다.

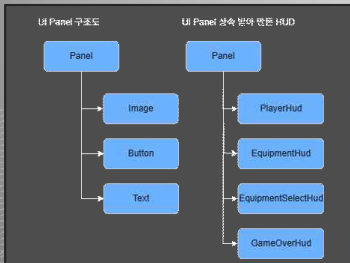
Equipment 현재



- 변경 후
 - Equipment -> Weapon -> Projectile, AOE
 - 장비별 기능 차이를 고려하여 상속 기반으로 구현하였다.
- 고찰점
 - 구현이 단순해졌다.
 - 장비별 책임이 명확하다.
 - 새로운 장비 추가가 쉬워졌다.
- 향후 개선 방향
 - 이번 프로젝트에서는 상속이 적합했다.
 - 하지만 프로젝트가 커질 경우 상속 계층이 계속 증가한다.
 - 다음 프로젝트에서는 공격, 상태이상, 효과, 판정 등을
 - 독립 Component로 분리하여 조합하는 구조로 적용해보고 싶다.

HoloCure Clone Project

- 프로젝트 구조도 - Equipment Select UI



● 개요

- 플레이어 레벨업 시 장비 후보를 생성하고, UI를 통해 플레이어가 장착 또는 업그레이드를 선택할 수 있도록 구성하였다.

● 동작 방식

- 1. 플레이어 레벨업 발생
- 2. EquipmentManager에서 장비 후보 벡터를 생성
- 3. UI Panel에 데이터를 전달
- 4. 선택시 해당 장비 장착 또는 해당 장비를 가지고 있는 경우 업그레이드를 진행

- 이미 보유한 장비일 경우 다음 레벨 효과를 표시하도록 구성하였다.

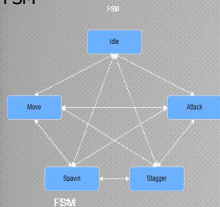
● 특징

- EquipmentManager가 선택 가능한 장비 목록을 생성하고,
- UI는 전달받은 데이터를 기반으로 선택 화면을 구성한다.
- 선택 결과에 따라 신규 장착 또는 기존 장비 업그레이드를 수행한다.

HoloCure Clone Project

- 프로젝트 구조도 - FSM & Camera

FSM



- 문제
- 상태 변경 코드가 여러 함수에 흩어져 있었다.
- 행동과 애니메이션이 동시에 변경되면서 상태 꼬임이 발생하였다.
- 새로운 상태 추가 시 여러 위치를 수정해야 했다.
- 해결
- SetState()를 통해 상태 변경을 한 곳에서만 수행하도록 변경하였다.
- 상태 전환 시 애니메이션도 함께 갱신하도록 구성하였다.

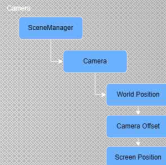
결과

- 상태 전환 흐름이 단순해졌다.
- 행동 충돌이 감소하였다.
- 상태 추가 및 유지보수가 쉬워졌다.

```
void Entity::SetState(State state)
{
    IF (_state == state)
        return;

    _state = state;
    UpdateAnimation();
}
```

Camera



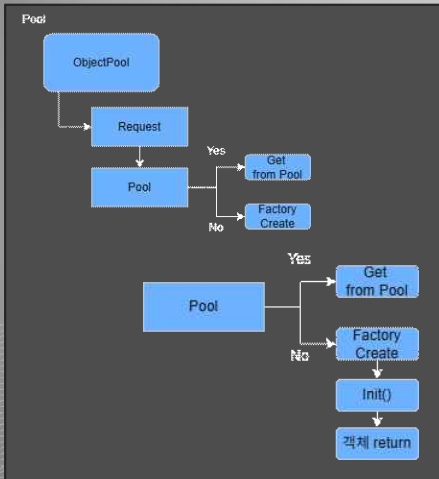
- 개요
- Camera를 기준으로 World 좌표를 Screen 좌표로 변환하여 렌더링하는 시스템
- 설계의도
- 맵 크기가 화면보다 커 Camera 추적이 필요하였다.
- 렌더러가 Camera를 신경쓰지 않도록 좌표 변환을 중앙에서 처리
- Camera가 월드 좌표를 화면 좌표로 변환하도록 하여 렌더러가 Camera를 직접 참조하지 않도록 설계하였다.
- 해결
- Camera를 독립 객체로 관리
- WorldToScreen / ScreenToWorld 함수 제공
- UI를 제외한 모든 렌더링이 Camera를 통해 좌표를 계산하도록 통일
- 기능
- 플레이어 추적
- 월드 좌표 -> 화면 좌표 변환
- 화면 좌표 -> 월드 좌표 변환

```
Vec2 SceneManager::WorldToScreen(Vec2 pos)
{
    return pos - _cameraPos + Vec2(GWInSizeX / 2, GWInSizeY / 2);
}

Vec2 SceneManager::ScreenToWorld(Vec2 pos)
{
    return pos + _cameraPos - Vec2(GWInSizeX / 2, GWInSizeY / 2);
}
```

HoloCure Clone Project

- 프로젝트 구조도 - Object Pool



```
// 오브젝트 풀에서 SausageObj를 꺼냄
TridentObj* tridentObj = GET_SINGLE(ObjectPool)->Get<TridentObj>(L"TridentObj");

inline T* ObjectPool::Get(wstring key)
{
    auto& pool = _ObjectPool[key];
    if (pool.empty())
    {
        CreateNewObject(key);
    }
    obj->Reset();
}
```

● 문제

- 투사체와 몬스터가 반복적으로 생성 / 삭제가 되었다.
- 생성과 삭제 비용이 지속적으로 발생하였다.

● 해결

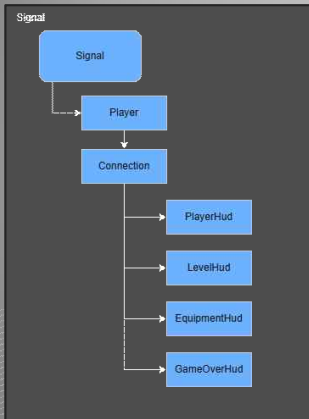
- Object Pool을 통해 비활성 객체를 재사용하였다.
- Pool이 비어있을 경우에만 새 객체를 생성하였다.
- 반환 시 Reset을 수행하여 초기 상태를 복원하였다.

● 결과

- 동적 할당 감소
- 생성 / 삭제 비용 감소
- 투사체, 몬스터 재사용

HoloCure Clone Project

- 프로젝트 구조도 - Signal



● 발행

```
OnExpChanged(_info._exp, _info._maxExp);
```

● 구독

```
_expConn =  
move(GET_SINGLE(GameManager)->GetPlayer()->OnExpChanged +=  
[this](float current, float max)  
{  
    _expBar->SetValue(current, max);  
});
```

● 문제

- Player가 UI를 직접 참조
- UI 증가 시 참조 증가
- 결합도 증가

● 해결

- Signal(Observer) 도입
- Player는 이벤트만 발생
- UI는 필요한 이벤트만 구독

● 장점

- Player는 UI를 직접 호출하지 않는다.
- UI는 필요한 이벤트만 구독
- UI 추가 시 Player 수정 없이 확장 가능
- 자동 Disconnect

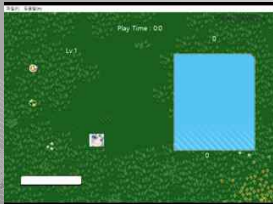
HoloCure Clone Project

- 프로젝트 구조도 - EditScene

UI 이동



저장된 데이터 로드



● 문제

- UI 배치 수정 시 매번 좌표를 직접 수정해야 했다.
- 수정 후 실행하여 결과를 확인해야 했다.
- 반복 작업으로 인해 작업 시간이 증가하였다.

● 해결

- UI를 Scene에서 직접 드래그하여 위치를 수정할 수 있도록 구현하였다.
- 수정한 UI Layout을 Key-Value 형태로 저장하였다.
- 게임 실행 시 저장된 데이터를 불러와 UI를 초기화하였다.

● 결과

- UI 위치 수정 후 즉시 결과 확인
- 반복 배치 작업 감소
- 레이아웃 재사용 가능
- UI 배치 작업을 코드 수정 없이 반복할 수 있도록 제작한 내부 툴이다.

● 기능

- 드래그 위치 조정
- Layout 저장
- Layout 불러오기
- Key-Value 기반 데이터 관리

HoloCure Clone Project

- 트러블 슈팅 - Signal

● 문제

- Signal 구독 시스템 구현 중 이벤트가 즉시 해제되는 문제가 발생
- 구독 직후 이벤트가 비정상적으로 끊어지는 현상 발생

● 원인 분석

- 처음에는 생명주기 문제라고 생각하였다.
- 그러나 디버깅 결과
- 직접 구현한 Connection 이동 생성자가
- 실제 이동을 수행하지 않고 디버그 로그만 찍고 있었다.

- 그래서 임시 객체가 소멸하여
- Disconnect()가 호출되어
- 구독이 제거되고 있었다.

```
Connection(Connection&&){
{
    cout << "[DEBUG] Connection 이동 생성" << endl;
}
```

● 배운점

- move semantics를 직접 구현할 경우
- 실제 자원 이동이 이루어지는지 반드시 확인해야한다.
- 컴파일러가 제공하는 기본 이동 생성자를
- 불필요하게 덮어쓰면 예상하지 못한 문제가 발생할 수 있다.

- 이번 경험을 통해 객체 소유권보다 이동 생성자의 동작을 정확히 이해하는 것이 중요하다는 것을 배웠다.

● 해결

- 이동 생성자를 제거하고
- 컴파일러가 제공하는
- 기본 이동 생성자를 사용하도록 변경하였다.
- Connection의 소유권이 정상적으로 이전되면서
- Disconnect가 호출되지 않도록 수정하였다.

```
Connection(Connection&&)=default;
```

HoloCure Clone Project

- 트러블 슈팅 - 게임 정지 상태에서 공격 발생

- 문제
- 게임 정지 상태(LevelUp UI)에서도 공격 객체가 생성되는 문제가 발생하였다.
- 플레이어는 조작할 수 없었지만 장비 시스템은 계속 동작하고 있었다.

● 증상

- LevelUp UI에서도 장비 공격이 계속 생성되었다.
- 일정 시간 선택하지 않으면 그동안 생성된 투사체가 한 번에 작동한다.
- 동일한 공격 객체가 여러 개 존재하는 현상 발생

● 원인

- EquipmentManager Update에서
- 게임 상태를 확인하지 않고 장비 업데이트를 수행하고 있었다.

● 그 결과

- LevelUp UI처럼 게임이 정지된 상태에서도 장비 쿨타임이 계속 진행되고 공격 객체가 생성되었다.

● 해결

```
if (GET_SINGLE(GameManager) -> GetState() != GameState::Playing)
    return;
```

- EquipmentManager가 Playing 상태에서만 Update를 수행하도록 수정하였다.

● 결과

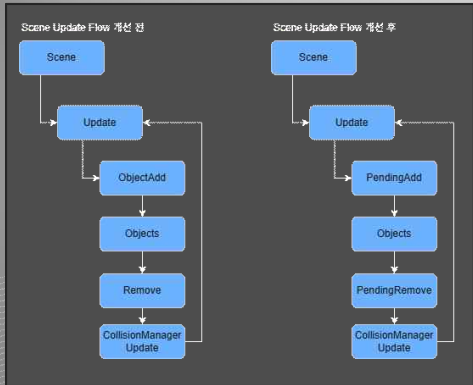
- 레벨업 중 공격 객체 생성 방지
- 의도하지 않은 투사체 누적 제거
- 게임 정지 상태와 실제 시스템 동작 일치

● 배운점

- GameState를 만드는 것보다 각 시스템이 현재 상태를 고려하여 동작 여부를 결정하는 것이 중요하다.
- 객체 내부 버그뿐 아니라 상위 시스템(GameState)와의 연동도 반드시 고려해야 한다.
- 시스템 간 책임과 실행 조건을 명확하게 설계해야 한다.

HoloCure Clone Project

- 트러블 슈팅 - Scene



● 문제

- Update 중 객체 추가 / 삭제가 발생한다.
- 순회 중 컨테이너가 변경되면서 Iterator 무효화 및 잘못된 객체 접근이 발생하였다.

● 해결

- 예약 시스템 도입
- PendingAdd(), PendingRemove() 를 두어
- Update 중에는 객체 생성 / 삭제를 수행하지 않고
- Update 종료 후 일괄 반영하도록 변경하였다.

● 결과

- Update 중 안전한 객체 생성 / 삭제
- Iterator 무효화 방지
- Scene 객체 생명주기 일관성 확보
- Scene 전환 시 객체 정리 단순화

HoloCure Clone Project

- 회고

- 프로젝트를 통해 얻은 것
 - Scene 기반 객체 관리
 - Object Pool, Signal(Event) 직접 구현
 - 객체 생명주기와 시스템 간 의존성 관리 경험
- 가장 큰 배움
 - 처음에는 Smart Pointer가 문제를 해결했다고 생각했다.
 - 하지만 실제 원인은 Move Semantics와 객체 소유권 이전이었다.
 - 디버깅을 통해 도구보다 객체 생명주기와 소유권 설계가 더 중요하다는 것을 배웠다.
- 아쉬운 점
 - 기능 구현을 우선하다 보니 장비 시스템이 상속 중심으로 커졌다,
 - 프로젝트 후반에는 Component 기반 분리와 데이터 기반 설계가 더 적합했음을 느꼈다.
- 앞으로 개선하고 싶은 점
 - Engine / Content Layer 분리
 - Component 기반 장비 기능 분리
 - Data Driven 구조 적용
 - Behavior Tree 등 AI 구조 개선

Project Cube

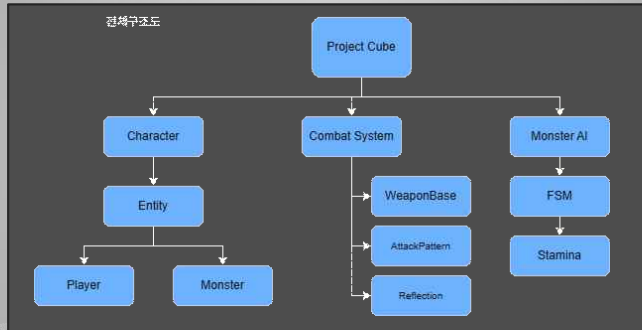
- 프로젝트 소개



- 프로젝트 소개
- Unity + C# 기반으로 제작한 Souls-like 액션 게임 프로젝트입니다.
- 플레이어 전투 시스템과 몬스터 AI를 구현하며 전투 시스템 설계와 상태 관리 방식을 학습하기 위해 진행했습니다.
- 주요 목표
 - Souls-like 전투 구현
 - AI 시스템 설계
 - 구조 설계 학습
- 개발 환경
 - Language : C#
 - Engine : Unity
- 주요 구현
 - Player
 - Monster AI
 - Boss Pattern
 - Combat System
- [플레이 영상](#)

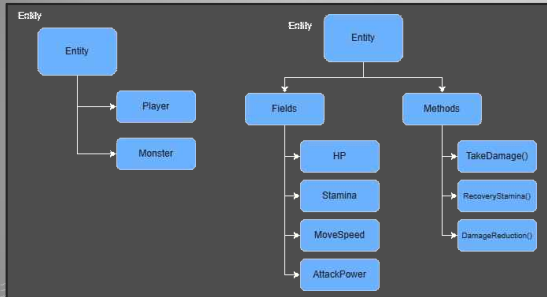
Project Cube

- 메인 구조도



Project Cube

- 플레이어 시스템 - Entity



- 개요
- Player와 Monster가 공통으로 사용하는 데이터와 기능을 관리하는 부모 클래스

- 설계 의도
- 공통 기능을 Entity에 정의하고
- Player와 Monster가 상속받도록 설계하였다.

- 이를 통해
- 중복 코드를 제거하고
- 객체별 기능만 구현하도록 분리하였다.

- 장점
- 공통 기능 재사용
- 새로운 Entity 추가 용이
- 객체별 기능만 구현

```
참조 10개
public virtual void TakeDamage(float damage, Vector3 hitPoint, Vector3 hitNormal) // 데미지 할수
참조 1개
public float DamageReduction(float damage, float decline_rate) // 데미지 감소를 적용된 데미지 계산
```

Project Cube

- 플레이어 시스템 구성

회피



플레이어
구성

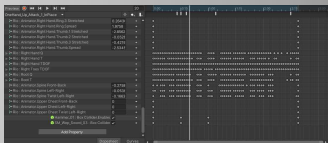


- 설계
- 입력 / 이동 / 공격 / 애니메이션 책임 분리
- Entity 상속으로 공통기능 재사용
- 컴포넌트별 역할 분리

Project Cube

- 전투 시스템 - WeaponBase - 공격 판정 제어

- 개요
- 무기의 공격 판정(Collider)을 제어하는 시스템
- 설계의도
- 초기에는 Coroutine에서 normalizedTime을 기준으로 공격 판정을 제어하였다.
- 하지만 애니메이션 길이가 변경될 때마다
- 코드의 normalizedTime도 함께 수정해야 했고,
- 애니메이션과 코드가 강하게 결합되어
- 유지보수가 어려웠다.
- 이를 개선하기 위해
- 공격 판정을 Animation Property에서
- 직접 제어하도록 변경하였다.



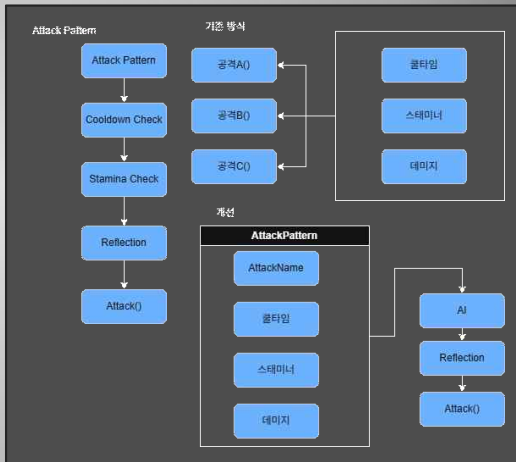
이전	개선
normalizedTime 기반	Animation Property 기반
코드 수정 필요	Animation 수정만으로 가능
타이밍 관리 어려움	애니메이션 중심 관리

Project Cube

- Monster AI - AttackPattern

- 개요
- 몬스터의 공격 정보와 실행에 필요한 데이터를 구조체로 관리하는 시스템이다.
- 설계 의도
- 초기에는 공격을 하나의 데이터처럼 관리할 수 있는 구조를 만들고자 했다.
- 각 공격마다 쿨타임, 스테미너, 호출 함수등의 정보가 함께 필요했기 때문에,
- 이를 하나의 구조체로 묶어 관리하는 방식을 고려하였다.
- 이후 공격 정보를 AttackPattern 구조체로 정의하고,
- 사용할 공격만 리스트에 등록하여 선택적으로 실행하는 구조를 설계하였다.
- 해결
- AI는 AttackPattern을 참조하여 공격 가능 여부를 판단한 뒤,
- Reflection으로 함수를 실행하도록 구성하였다.
- 장점
- 공격 데이터와 실행 로직을 분리하여 유지보수가 용이하다.
- 함수 재사용
- 공격 조합 변경 용이
- 신규 공격 추가 용이

```
public struct AttackPattern
{
    public string attackName;           // 공격 패턴 이름
    public float damage;                // 공격 데미지
    public float staminaCost;           // 소모 스테미너
    public float cooldownTime;          // 쿨타임
    [HideInInspector] public float lastUsedTime; // 마지막으로 사용 시간
}
```



Project Cube

- Monster AI - 문자열기반 함수 호출

● 개요

- 공격 정보를 하나의 데이터(AttackPattern)로 관리하고,
- 등록된 함수 정보를 이용해 필요한 공격을 실행하는 시스템이다.

● 설계 의도

- 공격마다 필요한 정보(쿨타임, 스테미너, 호출 함수)
- 하나의 구조체로 묶어 관리하고자 하였다.
- 공격을 리스트 형태로 관리함으로써 몬스터별로 사용할 공격을 조합할 수 있도록 설계하였다.
- 또한 공격 실행 로직을 공통화하여 동일한 처리 과정을 재사용할 수 있도록 하였다.

● 장점

- 함수 호출을 데이터로 관리
- 새로운 공격 추가 용이
- 공격 함수 재사용 가능
- 몬스터별 공격 조합 변경 용이

● 단점

- 문자열 오타 위험
- 컴파일 시 검사 불가
- 런타임 오류 가능성 증가
- 성능 부담

● 다시 구현하게 된다면

- Dictionary<AttackType, Action> 기반으로 하는게 좋을거 같다.



```
private void ExecuteAttack(int attackIndex)
{
    AttackPattern currentAttackPattern = attackPattern[attackIndex];
    _monsterBaseInfo.stamina -= currentAttackPattern.staminaCost; // 스테미너 차감

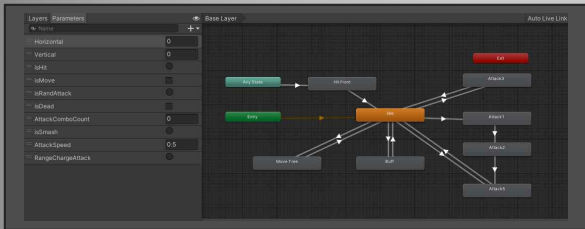
    Debug.Log($"{currentAttackPattern.attackName} 공격 실행!");

    // 실제 공격 로직 호출
    // (공격 이름, 공격이름의 함수이름이 같아야함)
    this.GetType().GetMethod(currentAttackPattern.attackName).Invoke(this, null);

    attackPattern[attackIndex].lastUsedTime = Time.time; // 마지막으로 공격한 시간 기록
}
```

Project Cube

- Monster AI - FSM



- 개요
 - 몬스터의 행동을 상태 단위로 관리하여 행동과 애니메이션을 제어하는 시스템이다.
- 설계 의도
 - 초기에는 조건문만으로 몬스터 행동을 제어하였다.
 - 공격, 이동, 피격 등이 동시에 발생하면서 행동 중첩 상태, 상태 꼬임 문제가 발생하였다.
 - 이를 해결하기 위해 현재 상태를 기준으로 가능한 행동만 수행하도록 FSM 구조를 적용하였다.

- 장점
 - 행동 충돌 감소
 - 상태별 애니메이션 제어
 - 상태 기반으로 AI 흐름 관리
 - 상태 전환 코드 단순화

Project Cube

- Monster AI - AI 흐름



● 개요

- 몬스터가 스태미너를 기반으로 공격과 휴식을 반복하며 전투 템포를 제어하는 AI 시스템이다.

● 설계 의도

- 공격만 반복하지 않고 스태미너를 소비, 회복하도록 구성
- 회복 시간을 만들어 플레이어에게 반격 기회를 제공
- 플레이어에게 공격 타이밍 제공

● 장점

- 공격 난이도 조절
- 전투 템포 형성
- 플레이어 반격 기회 제공



Project Cube

- 기술적 회고

당시 선택	다시 만든다면
Entity 중심 공통 관리	Component 확장 고려
Reflection	Dictionary<AttackType,Action>
Bool FSM	Enum + SetState
단순 AI	Behavior Tree

● Entity

- 당시 : Entity에는 체력, 데미지 처리등 공통 데이터와 기능만 관리하였다.
- 회고 : 공통 기능을 관리하는 구조는 적절했지만, 앞으로 기능이 증가한다면 부모 클래스를 확장하기보다 독립적인 기능은 Component로 분리하여 조합하는 방식이 더 유연할 것이라고 생각한다.

● Reflection

- 당시 : 공격 패턴을 빠르게 추가하기 위해 Reflection을 사용하였다.
- 회고 : 구현 속도는 빨랐지만 문자열 기반 호출이라 컴파일 검사가 불가능했다.
- 현재라면 : Dictionary<AttackType,Action>기반으로 구현하여 타입 안정성과 유지보수를 개선할 것이다.

● FSM

- 당시 : bool 변수로 상태를 관리하였다.
- 현재라면 : enum + SetState를 통해 상태 전환을 한 곳에서 관리하도록 설계할 것이다.

● AI

- 당시 : 조건문 기반으로 몬스터 행동을 구현했다.
- 현재라면 : Behavior Tree를 적용하여 확장성과 유지보수를 개선하겠다.